Diagrams

Calepin can run text-to-diagram engines from the same chunk system used for Python, R, and other computational code. Diagram chunks keep the source for figures in the Typst document, so prose, references, and diagram definitions stay together.

Diagram engines are stateless external tools. They convert source text to SVG, and *Calepin* renders the SVG as a figure. Give a diagram chunk a `label` and `fig-caption` when it should be numbered and cross-referenced.

Mermaid

Mermaid is a text-based diagramming tool; learn more at mermaid.js.org.

```
%%{init: {"htmlLabels": false}}%%
flowchart LR
  Source["Typst document"] --> Query["Calepin query"]
  Query --> Execute["Run chunks"]
  Execute --> Render["Typst render"]
```

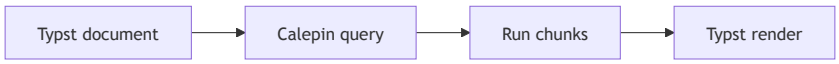

Figure 1: Mermaid flowchart

Graphviz DOT

Graphviz DOT is a graph description language used by Graphviz; learn more at graphviz.org.

```
digraph {
  rankdir=LR
  Draft -> Review -> Publish
  Review -> Draft [label="revise"]
}
```

Figure 2: Graphviz state graph

TikZ

TikZ is a LaTeX package for creating vector graphics; learn more at tikz.dev.

```
\begin{tikzpicture}
  \draw[thick, blue] (0,0) -- (2,1) -- (4,0);
  \fill[red] (2,1) circle (2pt);
\end{tikzpicture}
```

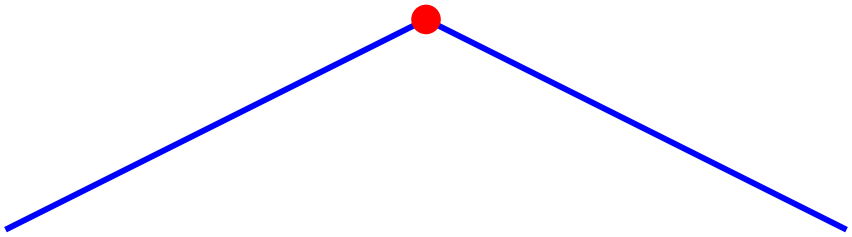

Figure 3: TikZ path

D2

D2 is a text-to-diagram language; learn more at d2lang.com.

```
direction: right

client -> api: request
api -> worker: job
worker -> database: write
```

Figure 4: D2 service sketch

Adding a new diagram engine

Diagram support is intentionally small. Each diagram engine is a stateless wrapper around an external command-line tool. *Calepin* writes the chunk source to a temporary file, asks the tool to produce SVG, and records that SVG as the figure output. Diagram engines do not keep a persistent session like Python, R, Julia, or shell chunks.

The main entry point is `calepin/src/engines/diagram.rs`. Add a `DiagramSpec` with:

- `name`: the fenced-code language users will write, such as `mermaid` or `dot`.
- `input_ext`: the temporary source-file extension passed to the external tool.
- `prepare_source`: usually `identity_source`, unless the tool needs a wrapper like TikZ.
- `render`: a function that runs the external tool and writes `run.fig_path`.

Simple tools can use the `simple_diagram_renderer!` helper in `diagram.rs`. More complex tools should get a small module under `calepin/src/engines/diagram/`, like `mermaid.rs` or `tikz.rs`, so retries, generated config files, or multi-step conversions stay out of the common path.

If the new engine uses a new executable, also register it in the surrounding plumbing:

- `calepin/src/config.rs` for the configurable executable path.
- `calepin/src/utils/tools.rs` for the default command name and install hint.
- `calepin/src/health/mod.rs` so `calepin health` can report whether the tool is available.
- `calepin/src/typst/preprocess/fingerprint.rs` so cached results are invalidated when the tool path changes.

Finally, make the chunk language visible to Typst and source rewriting by updating the built-in engine lists in `calepin/src/assets/typst-runtime/notebook/chunk.typ`, `calepin/src/typst/preprocess/staging.rs`, and any parser or execution tests that enumerate diagram engines. Add a small example to this page and tests for the renderer behavior.

Pull requests for new diagram engines are welcome. Open a pull request with the implementation, tests, and documentation example so the engine can be reviewed against the same notebook behavior as the built-in diagram tools.